

Task 1: Multi-node "Hello World" in MPI

1. Experimental Results

Configuration 1: 1 Node, 1 Task per Node

- **Result:**

Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch

- **Processes:** A single process ran on a single compute node.

Configuration 2: 1 Node, 2 Tasks per Node

- **Result:**

Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch
Hello World from rank: 1 on node: dmi-cl-node001.dmi.p.unibas.ch

- **Processes:** Two processes ran on the **same** node (shared memory context).

Configuration 3: 2 Nodes, 1 Task per Node

- **Result:**

Hello World from rank: 1 on node: dmi-cl-node001.dmi.p.unibas.ch
Hello World from rank: 0 on node: dmi-cl-node002.dmi.p.unibas.ch

- **Processes:** Two processes distributed across **two different** nodes.

Configuration 4: 2 Nodes, 2 Tasks per Node

- **Result:**

Hello World from rank: 1 on node: dmi-cl-node003...
Hello World from rank: 0 on node: dmi-cl-node003...

Hello World from rank: 2 on node: dmi-cl-node004...
Hello World from rank: 3 on node: dmi-cl-node004...

- **Processes:** Four processes distributed across two nodes (2 processes per node).

```
Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch
MASTER: Total number of MPI ranks is: 1
Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch
MASTER: Total number of MPI ranks is: 2
Hello World from rank: 1 on node: dmi-cl-node001.dmi.p.unibas.ch
Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch
Hello World from rank: 1 on node: dmi-cl-node002.dmi.p.unibas.ch
MASTER: Total number of MPI ranks is: 2
Hello World from rank: 1 on node: dmi-cl-node003.dmi.p.unibas.ch
Hello World from rank: 0 on node: dmi-cl-node003.dmi.p.unibas.ch
Hello World from rank: 2 on node: dmi-cl-node004.dmi.p.unibas.ch
MASTER: Total number of MPI ranks is: 4
Hello World from rank: 3 on node: dmi-cl-node004.dmi.p.unibas.ch
```

2. Discussion

Question: If our goal of using MPI is to exchange messages between nodes, which job configurations make sense and which don't?

Answer:

- **Configurations 1.1 and 1.2 (Single Node)** do not utilize the primary strength of MPI, which is distributed computing across a network. In these cases, communication happens over shared memory (RAM) within a single machine. While useful for debugging, they do not represent a distributed system scenario.
- **Configurations 1.3 and 1.4 (Multi-Node)** are the configurations that make sense for MPI. They force communication across the network interconnect (Inter-node communication), which is the standard use case for High-Performance Computing (HPC) clusters.

Task 2: Blocking MPI Send and Receive

1. Experimental Results

Configuration: 4 Nodes, 1 Task per Node (Total 4 Ranks)

- **Result:**

```
[uygun0001@dm1-cl-login mpi_exercise]$ cat out_task2_*.txt
Rank 0: Sent message to Rank 1
Rank: 1 on node: dm1-cl-node002.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch
Rank 0: Sent message to Rank 2
Rank: 2 on node: dm1-cl-node003.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch
Rank 0: Sent message to Rank 3
Rank: 3 on node: dm1-cl-node004.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch
```

Rank 0: Sent message to Rank 1
Rank: 1 on node: dm1-cl-node002... received... Hello World from rank: 0...
Rank 0: Sent message to Rank 2
Rank: 2 on node: dm1-cl-node003... received... Hello World from rank: 0...
Rank 0: Sent message to Rank 3
Rank: 3 on node: dm1-cl-node004... received... Hello World from rank: 0...

- **Analysis:**

The output confirms the **blocking** nature of `MPI_Send`. Rank 0 must complete the send operation to Rank 1 before it can begin sending to Rank 2. This results in a strictly sequential execution order.

2. Discussion

Question: What happens if you increase `ntasks-per-node`? Explain the resulting behavior.

Experiment: We increased `ntasks-per-node` to **2** (keeping `nodes=4`), resulting in a total of **8 Ranks**.

```
[uygun0001@dm1-cl-login mpi_exercise]$ cat out_task2_ntasks2_*.txt
Rank 0: Sent message to Rank 1
Rank: 1 on node: dm1-cl-node001.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch
Rank 0: Sent message to Rank 2
Rank: 2 on node: dm1-cl-node002.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch
Rank 0: Sent message to Rank 3
Rank: 3 on node: dm1-cl-node002.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch
Rank 0: Sent message to Rank 4
Rank: 4 on node: dm1-cl-node003.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch
Rank 0: Sent message to Rank 5
Rank: 5 on node: dm1-cl-node003.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch
Rank 0: Sent message to Rank 6
Rank: 6 on node: dm1-cl-node004.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch
Rank 0: Sent message to Rank 7
Rank: 7 on node: dm1-cl-node004.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch
```

Output:

Rank 0: Sent message to Rank 1
Rank: 1 on node: dm1-cl-node001.dmi.p.unibas.ch received the following message:
Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch

Rank 0: Sent message to Rank 2
Rank: 2 on node: dm1-cl-node002.dmi.p.unibas.ch received the following message:
Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch

Rank 0: Sent message to Rank 3
Rank: 3 on node: dm1-cl-node002.dmi.p.unibas.ch received the following message:
Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch

Rank 0: Sent message to Rank 4
Rank: 4 on node: dm1-cl-node003.dmi.p.unibas.ch received the following message:
Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch

Rank 0: Sent message to Rank 5
Rank: 5 on node: dm1-cl-node003.dmi.p.unibas.ch received the following message:
Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch

Rank 0: Sent message to Rank 6
Rank: 6 on node: dm1-cl-node004.dmi.p.unibas.ch received the following message:
Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch

Rank 0: Sent message to Rank 7
Rank: 7 on node: dm1-cl-node004.dmi.p.unibas.ch received the following message:
Hello World from rank: 0 on node: dm1-cl-node001.dmi.p.unibas.ch

Explanation of Behavior: Increasing ntasks-per-node resulted in two significant behaviors:

- **Increased Serialization Delay:** The sender (Rank 0) now has to communicate with 7 receivers instead of 3. Since blocking communication is used, the sender is "stuck" 7 times sequentially. This serialization creates a performance bottleneck that scales linearly with the number of ranks.

Heterogeneous Communication Costs: The system now exhibits mixed communication latencies:

- **Intra-node:** Rank 0 communicating with Rank 1 (on the same node) uses shared memory (Low Latency).
- **Inter-node:** Rank 0 communicating with Rank 2 (on a different node) uses the network (High Latency).
- *Impact:* Even though intra-node communication is fast, the blocking nature forces the sender to wait for slower network operations (inter-node) before it can finish its entire loop.

Task 3: Non-blocking MPI Send and Receive

1. Experimental Results

Configuration: 4 Nodes, 1 Task per Node (Total 4 Ranks).

Result:

```
[uygun001@dmi-cl-login mpi_exercise]$ cat out_task3_*.txt
Rank 0: Initiating non-blocking send to Rank 1
Rank 1: Posting non-blocking receive...
Rank 3: Posting non-blocking receive...
Rank 2: Posting non-blocking receive...
Rank 0: Initiating non-blocking send to Rank 2
Rank 1: Receive posted, now doing other work...
Rank 3: Receive posted, now doing other work...
Rank 2: Receive posted, now doing other work...
Rank 1: Waiting for message to arrive...
Rank 3: Waiting for message to arrive...
Rank 0: Initiating non-blocking send to Rank 3
Rank 2: Waiting for message to arrive...
Rank: 1 on node: dmi-cl-node002.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch
Rank: 2 on node: dmi-cl-node003.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch
Rank 0: All sends initiated, now doing other work...
Rank: 3 on node: dmi-cl-node004.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch
Rank 0: Waiting for all sends to complete...
Rank 0: All sends completed successfully.
```

Rank 0: Initiating non-blocking send to Rank 1
Rank 1: Posting non-blocking receive...
Rank 3: Posting non-blocking receive...
Rank 2: Posting non-blocking receive...
Rank 0: Initiating non-blocking send to Rank 2
Rank 1: Receive posted, now doing other work...
Rank 3: Receive posted, now doing other work...
Rank 2: Receive posted, now doing other work...
Rank 1: Waiting for message to arrive...
Rank 3: Waiting for message to arrive...
Rank 0: Initiating non-blocking send to Rank 3
Rank 2: Waiting for message to arrive...

Rank: 1 on node: dmi-cl-node002.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch

Rank: 2 on node: dmi-cl-node003.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch

Rank 0: All sends initiated, now doing other work...

Rank: 3 on node: dmi-cl-node004.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch

Rank 0: Waiting for all sends to complete...

Rank 0: All sends completed successfully.

2. Discussion

Question: What do you notice in the ordering of the output messages?

Observation: The output ordering is **interleaved and asynchronous**.

1. Rank 0 initiates sends to Rank 1, 2, and 3 in quick succession without waiting for acknowledgment.
2. The message *"All sends initiated, now doing other work..."* appears *before* Rank 0 waits for completion.
3. Receivers print *"Receive posted, now doing other work..."* before the message actually arrives. This proves that the CPU was free to execute other instructions while the MPI library handled the network communication in the background.

Question: Under which assumptions is non-blocking communication better than blocking communication?

Answer: Non-blocking communication is superior under the assumption that **computation and communication can be overlapped (latency hiding)**.

- **Blocking:** The CPU sits idle while data travels over the network.
- **Non-blocking:** The CPU issues a "send" command and immediately returns to perform useful calculations (physics updates, data processing, etc.) while the network hardware handles the transfer. If the time taken for computation is less than or equal to the communication time, the communication cost is effectively "hidden."

Task 4: Collective Communication with MPI Broadcast

1. Experimental Results

Configuration: 4 Nodes, 1 Task per Node.

▪ Result:

```
[uygun001@dmi-cl-login mpi_exercise]$ cat out_task4_*.txt
Rank 0: Broadcasting message to all ranks...
Rank: 2 on node: dmi-cl-node003.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch
Rank 0: Broadcast completed successfully.
Rank: 3 on node: dmi-cl-node004.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch
Rank: 1 on node: dmi-cl-node002.dmi.p.unibas.ch received the following message: Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch
```

Rank 0: Broadcasting message to all ranks...

Rank: 2 on node: dmi-cl-node003.dmi.p.unibas.ch received the following message:
Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch

Rank 0: Broadcast completed successfully.

Rank: 3 on node: dmi-cl-node004.dmi.p.unibas.ch received the following message:
Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch

Rank: 1 on node: dmi-cl-node002.dmi.p.unibas.ch received the following message:
Hello World from rank: 0 on node: dmi-cl-node001.dmi.p.unibas.ch

2. Discussion

Question: How does this approach compare to blocking/non-blocking communication? What are the advantages/disadvantages?

Comparison:

- **Point-to-Point (Blocking/Non-blocking):** Requires the sender to manually transmit $N-1$ messages. The complexity is linear, $O(N)$. The sender is the bottleneck.
- **Collective (MPI Bcast):** The MPI library optimizes the data distribution, typically using tree-based algorithms (e.g., Binomial Tree). Rank 0 sends to two nodes, those two send to two others, and so on. The complexity is logarithmic, $O(\log N)$.

Advantages:

1. **Scalability:** On large clusters, MPI_Bcast is significantly faster than a loop of Sends because the bandwidth load is distributed across the network rather than concentrated on the Root node.
2. **Code Readability:** It reduces complex loop logic and error handling into a single function call, minimizing the chance of programming errors (like deadlocks from mismatched tags).
3. **Optimization:** The underlying MPI implementation automatically selects the best algorithm based on message size and topology.

Disadvantages:

1. **Rigidity:** MPI_Bcast requires sending the **exact same data** to everyone. If Rank 1 needs "Message A" and Rank 2 needs "Message B", MPI_Bcast cannot be used efficiently; point-to-point (Scatter) would be needed.
2. **Implicit Synchronization:** Collective operations often act as synchronization points. If one rank is late reaching the MPI_Bcast call, other ranks may have to wait, potentially reducing overall parallel efficiency if the load is unbalanced.

Task 5: Synchronization with MPI Barrier

Analysis of Barriers

We analyzed the five instances where MPI_Barrier is called in the code:

1. Line 7: MPI_Barrier(MPI_COMM_WORLD);

- **Location:** Before MPI_Init.
- **Classification:** **Produces an error (Runtime Error).**
- **Reasoning:** MPI functions cannot be called before the MPI execution environment is initialized. Since MPI_Init has not been called yet, the communicator MPI_COMM_WORLD does not exist, causing the program to crash immediately.

2. Line 9: MPI_Barrier(MPI_COMM_WORLD);

- **Location:** Immediately after MPI_Init.
- **Classification:** **Implementation-wise OK, but useless/illogical.**
- **Reasoning:** While syntactically correct (as MPI is now initialized), this barrier serves no logical purpose. The processes have just started, and no computation, I/O, or communication has occurred that requires synchronization. It adds unnecessary overhead.

3. Lines 19 & 20: Conditional Barrier Logic

- **Location:**
 - Line 19: if (rank == 0) { MPI_Barrier(...); }.
 - Line 20: MPI_Barrier(...); (Outside the if block).
- **Classification:** **Produces an error (Deadlock).**
- **Reasoning:** MPI_Barrier is a collective operation, meaning **all** processes in the communicator must call it.
 - **Rank 0** enters the if block and waits at the barrier on **Line 19**.
 - **Other Ranks** skip the if block and wait at the barrier on **Line 20**.
 - Because Rank 0 is waiting at a different point in the code logic than the other ranks, they are effectively waiting for a matching call that will never happen. The program hangs indefinitely (Deadlock).

4. Line 25: MPI_Barrier(MPI_COMM_WORLD);

- **Location:** Immediately after MPI_Bcast.
- **Classification:** **Implementation-wise OK, but useless/illogical.**
- **Reasoning:** MPI_Bcast acts as a blocking operation for the receivers. When it returns, the process is guaranteed to have received the data. Adding an explicit barrier right after a blocking collective operation is redundant and provides no additional safety.

5. Line 31: `MPI_Barrier(MPI_COMM_WORLD);`

- **Location:** After `MPI_Finalize`.
- **Classification:** **Produces an error (Runtime Error).**
- **Reasoning:** `MPI_Finalize` shuts down the MPI environment and disconnects the processes. Calling any MPI function (including a barrier) after finalization is illegal and results in a runtime error.

Conclusion

The code provided in Listing 2 is heavily flawed. It attempts to use MPI functions outside the active lifecycle of the MPI environment (Lines 7 and 31) and contains a logic error that guarantees a deadlock (Lines 19/20). The remaining barriers provide no functional benefit to the program's execution.